

Kafka Producer/Consumer Reference

Fallback reference only — write your own logic in the real files first. This is what a complete, working version could look like, file by file, across both repos. Field names/topic names here are suggestions; keep both sides in sync if you change them.

Topics used: `transaction.completed`, `beneficiary.activated`

Side A — Enterprise-Digital-Banking-System (producer)

`pom.xml` (add dependency)

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

`src/main/resources/application.properties` (add)

```
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
```

`docker-compose.yml` (local single-node Kafka, KRaft mode, no Zookeeper needed)

```
version: "3.8"
services:
  kafka:
    image: apache/kafka:3.8.0
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Run with `docker compose up -d`, check it's listening with `docker ps` / `docker logs kafka`.

`kafka/event/TransactionCompletedEvent.java`

```

package org.example.enterprisedigitalbankingsystem.kafka.event;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Getter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class TransactionCompletedEvent {
    private String eventId;           // UUID.randomUUID().toString() - becomes the idempotencyKey down
    private Long transactionId;
    private String transactionReference;
    private String transactionType;   // DEPOSIT / WITHDRAWAL / TRANSFER
    private Long accountId;
    private String accountNumber;
    private Long customerId;
    private String userId;           // User.getUserId().toString()
    private String email;
    private BigDecimal amount;
    private BigDecimal balanceAfterTransaction;
    private LocalDateTime occurredAt;
}

```

kafka/event/BeneficiaryActivatedEvent.java

```
package org.example.enterprisedigitalbankingsystem.kafka.event;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Getter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class BeneficiaryActivatedEvent {
    private String eventId;
    private Long beneficiaryId;
    private Long customerId;
    private String userId;
    private String email;
    private String nickName;
    private String accountNumber;
    private LocalDateTime occurredAt;
}
```

kafka/producer/TransactionEventProducer.java

```

package org.example.enterprisedigitalbankingsystem.kafka.producer;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.enterprisedigitalbankingsystem.kafka.event.TransactionCompletedEvent;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
@Slf4j
public class TransactionEventProducer {

    private static final String TOPIC = "transaction.completed";

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    public void publish(TransactionCompletedEvent event) {
        try {
            String payload = objectMapper.writeValueAsString(event);
            kafkaTemplate.send(TOPIC, event.getAccountId().toString(), payload);
        } catch (Exception e) {
            // Don't let a Kafka/serialization failure roll back the transaction itself.
            log.error("Failed to publish TransactionCompletedEvent for transaction {}",
                event.getTransactionId(), e);
        }
    }
}

```

`event.getAccountId().toString()` as the Kafka **key** keeps all events for the same account in the same partition (so ordering is preserved per account).

`kafka/producer/BeneficiaryEventProducer.java`

```

package org.example.enterprisedigitalbankingsystem.kafka.producer;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.enterprisedigitalbankingsystem.kafka.event.BeneficiaryActivatedEvent;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
@Slf4j
public class BeneficiaryEventProducer {

    private static final String TOPIC = "beneficiary.activated";

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    public void publish(BeneficiaryActivatedEvent event) {
        try {
            String payload = objectMapper.writeValueAsString(event);
            kafkaTemplate.send(TOPIC, event.getCustomerId().toString(), payload);
        } catch (Exception e) {
            log.error("Failed to publish BeneficiaryActivatedEvent for beneficiary {}",
                    event.getBeneficiaryId(), e);
        }
    }
}

```

Wiring – snippets only, not full files

In `TransactionServiceImpl.transfer()`, after `ledgerService.recordTransactionEntries(transaction)`:

```

transactionEventProducer.publish(TransactionCompletedEvent.builder()
    .eventId(UUID.randomUUID().toString())
    .transactionId(transaction.getId())
    .transactionReference(transaction.getTransactionReference())
    .transactionType(transaction.getTransactionType().name())
    .accountId(sourceAccount.getId())
    .accountNumber(sourceAccount.getAccountNumber())
    .customerId(sourceAccount.getCustomer().getId())
    .userId(sourceAccount.getCustomer().getUser().getUserId().toString()) // adjust to your actual Customer
    .email(sourceAccount.getCustomer().getEmail())
    .amount(amount)
    .balanceAfterTransaction(sourceNewBalance)
    .occurredAt(LocalDate.now())
    .build());

```

In `BeneficiaryServiceImpl.activateBeneficiary()`, after saving the status change:

```
beneficiaryEventProducer.publish(BeneficiaryActivatedEvent.builder()  
    .eventId(UUID.randomUUID().toString())  
    .beneficiaryId(beneficiary.getId())  
    .customerId(beneficiary.getCustomer().getId())  
    .userId(beneficiary.getCustomer().getUser().getUserId().toString())  
    .email(beneficiary.getCustomer().getEmail())  
    .nickName(beneficiary.getNickName())  
    .accountNumber(beneficiary.getBeneficiaryAccount().getAccountNumber())  
    .occurredAt(LocalDateTime.now())  
    .build());
```

Side B — notification-service (consumer)

Kafka dependency and `spring.kafka.*` config already exist in its `pom.xml` / `application.yml` — nothing to add there.

`event/TransactionCompletedEvent.java`

```
package org.example.notification.service.event;  
  
import java.math.BigDecimal;  
import java.time.LocalDateTime;  
  
public record TransactionCompletedEvent(  
    String eventId,  
    Long transactionId,  
    String transactionReference,  
    String transactionType,  
    Long accountId,  
    String accountNumber,  
    Long customerId,  
    String userId,  
    String email,  
    BigDecimal amount,  
    BigDecimal balanceAfterTransaction,  
    LocalDateTime occurredAt) {  
  
}
```

`event/BeneficiaryActivatedEvent.java`

```
package org.example.notification.service.event;

import java.time.LocalDateTime;

public record BeneficiaryActivatedEvent(
    String eventId,
    Long beneficiaryId,
    Long customerId,
    String userId,
    String email,
    String nickName,
    String accountNumber,
    LocalDateTime occurredAt) {
}
```

These two records must mirror the producer's field names/types — they're two independent classes in two independent repos, kept in sync by convention, not by a shared library.

consumer/TransactionEventConsumer.java

```

package org.example.notification.service.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.notification.service.dto.request.NotificationRequest;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationType;
import org.example.notification.service.event.TransactionCompletedEvent;
import org.example.notification.service.service.NotificationService;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
@Slf4j
public class TransactionEventConsumer {

    private final NotificationService notificationService;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = "transaction.completed", groupId = "notification-service")
    public void onTransactionCompleted(String payload) {
        try {
            TransactionCompletedEvent event = objectMapper.readValue(payload, TransactionCompletedEvent.class);

            notificationService.send(new NotificationRequest(
                event.userId(),
                "Transaction Successful",
                "Your " + event.transactionType() + " of " + event.amount()
                    + " on account " + event.accountNumber() + " was successful. "
                    + "New balance: " + event.balanceAfterTransaction(),
                NotificationType.EMAIL,
                NotificationPriority.HIGH,
                event.eventId() // reused as the idempotency key
            ));
        } catch (Exception e) {
            log.error("Failed to process TransactionCompletedEvent payload: {}", payload, e);
        }
    }
}

```

consumer/BeneficiaryEventConsumer.java

```

package org.example.notification.service.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.example.notification.service.dto.request.NotificationRequest;
import org.example.notification.service.enums.NotificationPriority;
import org.example.notification.service.enums.NotificationType;
import org.example.notification.service.event.BeneficiaryActivatedEvent;
import org.example.notification.service.service.NotificationService;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
@Slf4j
public class BeneficiaryEventConsumer {

    private final NotificationService notificationService;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = "beneficiary.activated", groupId = "notification-service")
    public void onBeneficiaryActivated(String payload) {
        try {
            BeneficiaryActivatedEvent event = objectMapper.readValue(payload, BeneficiaryActivatedEvent.class);

            notificationService.send(new NotificationRequest(
                event.userId(),
                "Beneficiary Activated",
                "Your beneficiary \"" + event.nickName() + "\" (" + event.accountNumber()
                    + ") is now active and ready for transfers.",
                NotificationType.EMAIL,
                NotificationPriority.MEDIUM,
                event.eventId()
            ));
        } catch (Exception e) {
            log.error("Failed to process BeneficiaryActivatedEvent payload: {}", payload, e);
        }
    }
}

```

Notes / known gaps

- `NotificationRequest.userId` is a plain string over there with no lookup — these events carry `email` too, for whenever that service wires up real delivery (right now `EmailNotificationChannel` /etc. just log, they don't actually send).
- The producer swallows Kafka/serialization errors (logs only) rather than throwing — a notification failure should never roll back a completed bank transaction.
- Topic names (`transaction.completed` , `beneficiary.activated`) must match exactly between `@KafkaListener(topics = "...")` here and whatever string you `kafkaTemplate.send(...)` with on

the producer side.